Octobre 2026

Rapport de TP

Ingerer les donnees simulees & Rechercher (SOC L1)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Ethical Hacking / SOC

Rapport de travaux pratiques : TP 2, Ingerer les donnees simulees et rechercher. Parcours SaaS : template ECS, upload de datasets, Data View, KQL, cle API.



Republique du Senegal, Un Peuple, Un But, Une Foi
Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation
Direction Generale de l'Enseignement Superieur Prive
Institut Superieur de Management
Annee academique 2026-2027

Table des matieres

Partie I : Presentation du travail....3
Partie II : Appliquer le template ECS....4
Partie III : Uploader les datasets....6
Partie IV : Verifier l'ingestion (Dev Tools)....10
Partie V : Creer la Data View....13
Partie VI : Recherche KQL....14
Partie VII : Cle API restreinte....18
Partie VIII : Defis (autonomie)....22
Conclusion....27

Partie I : Présentation du travail

Contexte

A l'issue du TP1, la plateforme SOC-LAB-GLACIER (Elastic Cloud Hosted) est opérationnelle mais vide : aucun index de données n'existe encore. Ce TP2 enchaîne directement dessus avec pour objectif d'injecter des événements ECS simules (trafic normal et scénarios d'attaque, livres sous forme de fichiers NDJSON) puis de commencer a les interroger. Les mêmes datasets que le parcours auto-hébergé sont utilises, l'objectif étant de comparer les deux méthodes d'ingestion plutôt que le contenu des données lui-même.

Objectifs pédagogiques

- Appliquer un template d'index ECS pour garantir des mappings corrects.
- Ingerer des datasets NDJSON via Kibana, sans outil local.
- Vérifier l'ingestion dans Dev Tools (index, volumetrie, mappings).
- Créer une Data View et l'utiliser dans Discover.
- Ecrire des requêtes KQL et borner le temps.
- Créer une cle API restreinte (moindre privilege).
- Identifier des IOC a partir des événements.

Datasets fournis

Fichier	Événements	Contenu
sample-normal.ndjson	300	Trafic legitime uniquement
sample-mixed.ndjson	1000	85% normal / 10% suspect / 5% malveillant
scénario-brute-force.ndjson	60	Echecs SSH repetes
scénario-port-scan.ndjson	80	Scan multi-ports (pare-feu)
scénario-web-attack.ndjson	60	SQLi, path traversal, User-Agent offensif
tp5-incident.ndjson	~1195	Kill chain complète, reservee au TP5

Conformement a la consigne, les cinq premiers datasets (hors tp5-incident.ndjson, reserve au TP5) ont été uploades, pour un total attendu de 1500 événements (300 + 1000 + 60 + 80 + 60).

Point de contrôle 1

Chaque dataset a été associe a son scénario avant l'ingestion : trafic normal (sample-normal, sample-mixed), brute force SSH, scan de ports, et attaque web. Le fichier tp5-incident.ndjson a été volontairement laisse de côté pour le TP5.

Partie II : Appliquer le template ECS

Pourquoi un template avant l'ingestion

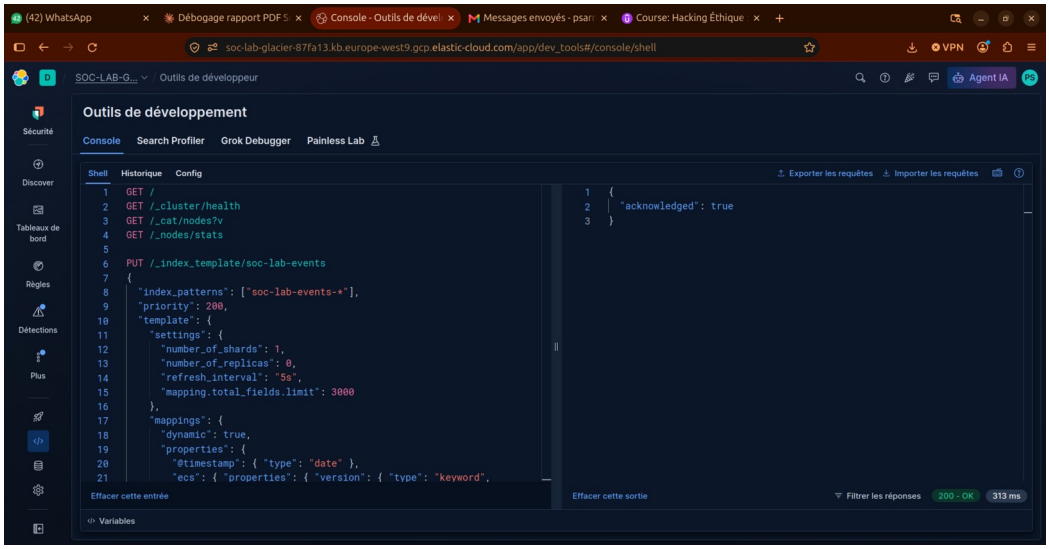
Un template d'index force les bons types (keyword, ip, date) au moment de la création d'un index correspondant a son pattern. Sans lui, un upload avec inference automatique de mapping typerait la plupart des champs en text, ce qui casse les agregations exactes, les recherches par plage sur les adresses IP (CIDR) et, plus généralement, fiabilise moins les règles de détection construites plus tard (TP4).

Application du template

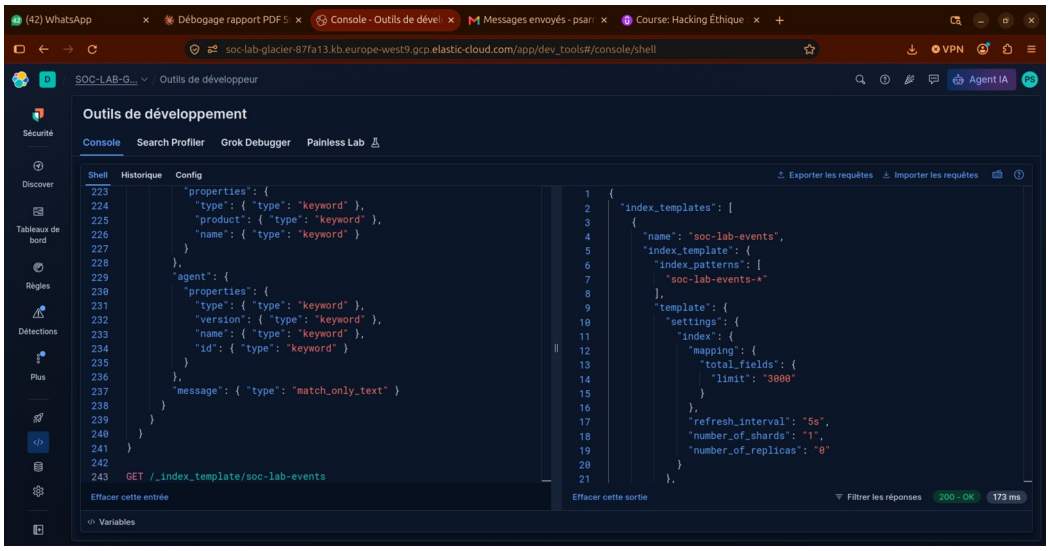
Le contenu complet du fichier index-template.json (236 lignes, mappings ECS couvrant @timestamp, event.*, host.*, user.*, source.*/destination.*, network.*, process.*, file.*, url.*, http.*, rule.*, dns.*, registry.*, winlog.*, log.*, service.*, observer.*, agent.* et message en match_only_text) a été colle dans Dev Tools et execute avec la commande suivante :

```
PUT /_index_template/soc-lab-events
{
  "index_patterns": ["soc-lab-events-*"],
  "priority": 200,
  "template": { "settings": { ... }, "mappings": { ... } }
}
```

La réponse a confirme l'application du template : {"acknowledged": true}, code 200 OK. Une verification immédiate a été effectuée avec GET /_index_template/soc-lab-events, qui a renvoye 200 OK avec l'integralite du contenu colle, confirmant que le template est bien enregistre côté cluster avant toute ingestion.



Dev Tools : PUT /_index_template/soc-lab-events, réponse {"acknowledged": true}, 200 OK en 313 ms.



Dev Tools : GET /_index_template/soc-lab-events, le template enregistre est renvoye en integralite (index_patterns, settings, mappings), 200 OK en 173 ms.

Signification de index_patterns (Q4.1)

La valeur ["soc-lab-events-*"] indique qu'Elasticsearch appliquera automatiquement ce template a tout index cree dont le nom commence par soc-lab-events-, quel que soit le suffixe utilise ensuite (une date comme 2026.10.18, un nom libre, etc.). C'est ce mécanisme de correspondance par motif qui garantit que chaque nouvel index de données du lab, même cree via l'assistant d'upload sans intervention manuelle, herite automatiquement des bons mappings.

Pourquoi source.ip en type ip plutôt que text (Q4.2)

Le type ip permet a Elasticsearch de comprendre la semantique d'une adresse IPv4/IPv6 : il autorise les requêtes par plage et par notation CIDR (par exemple source.ip: "10.10.0.0/16"), des agregations exactes (Top values, cardinalite) et un stockage compact. Un mapping text, au contraire, decoupe la chaîne en tokens separes par les points (tokenisation standard), ce qui detruit complètement la structure de l'adresse : une recherche sur une plage ou un sous-réseau devient impossible, et les valeurs distinctes ne sont plus comptees correctement.

Consequence de l'absence du template a l'upload (Q4.3)

Sans template prealable, le premier document uploade declencherait l'inference de mapping dynamique d'Elasticsearch : les champs numeriques et les adresses IP seraient très probablement types en text ou en champs multiples non adaptes (selon les heuristiques d'inference de l'assistant d'upload), et ce mapping, une fois fixe par le premier document indexe, s'appliquerait a tous les documents suivants du même index. Corriger un mapping errone après coup nécessite de reindexer entièrement les données dans un nouvel index, une opération coûteuse evitee ici en appliquant le template avant tout upload.

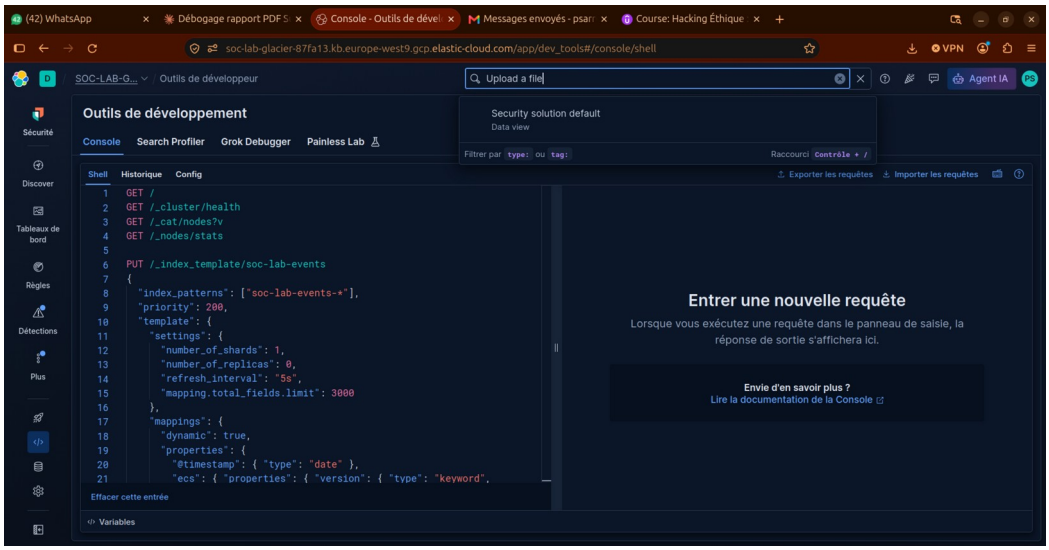
Point de contrôle 2

Le template soc-lab-events existe et répond 200 OK a la requête GET /_index_template/soc-lab-events, avec l'integralite des mappings ECS attendus.

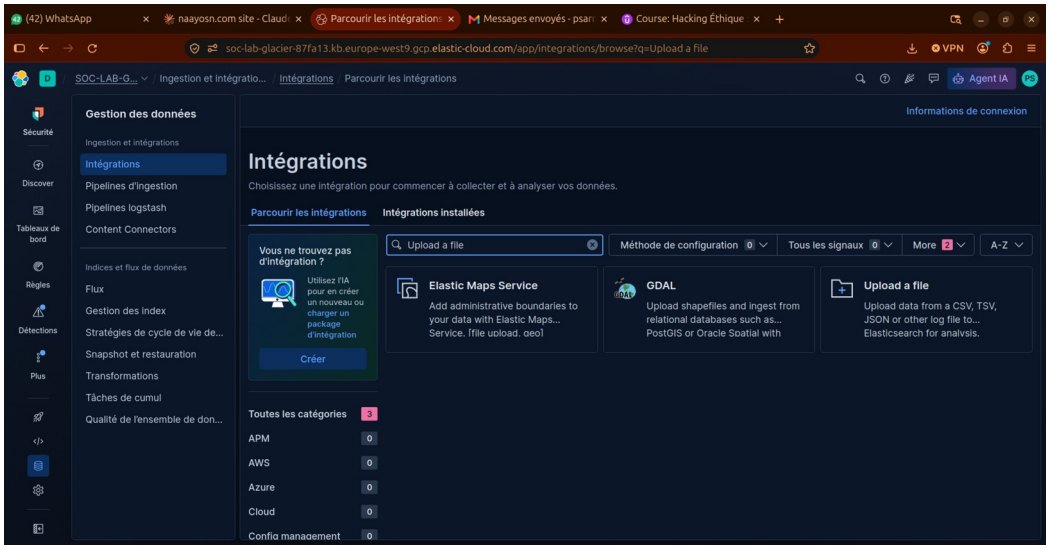
Partie III : Uploader les datasets

Accès a l'outil d'upload

La fonctionnalité Upload a file (Data Visualizer) n'a pas été trouvée via la barre de recherche globale de Kibana : taper Upload a file n'a remonte qu'un résultat sans rapport (une Data View existante). L'accès a été obtenu en naviguant directement vers /app/integrations/browse, ou la carte Upload a file est effectivement présente et accessible.



Barre de recherche globale de Kibana : la saisie "Upload a file" ne remonte qu'une Data View existante, sans rapport avec l'outil recherche.

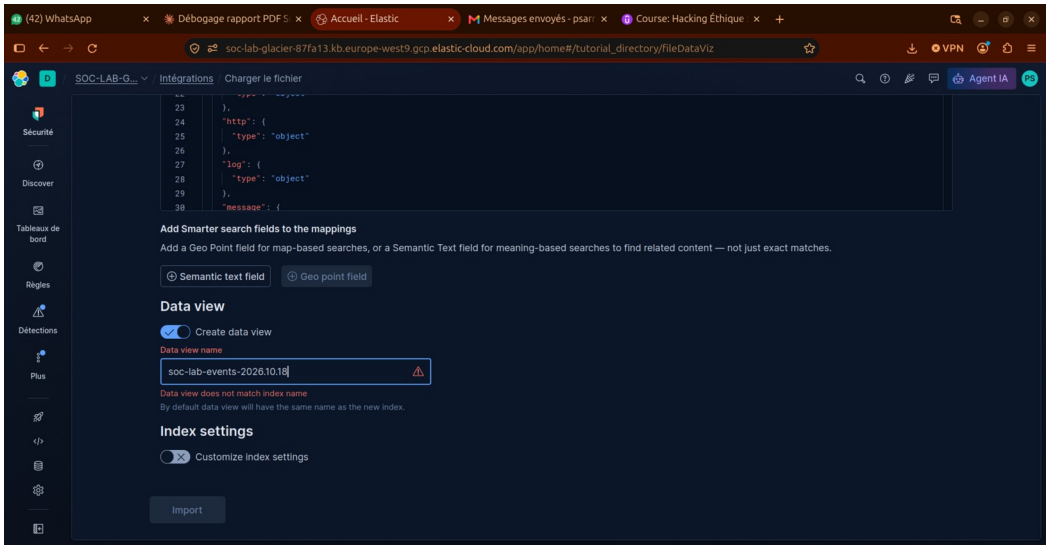


Page Parcourir les integrations (/app/integrations/browse) : la carte "Upload a file" (données CSV, TSV, JSON...) y est bien présente et accessible.

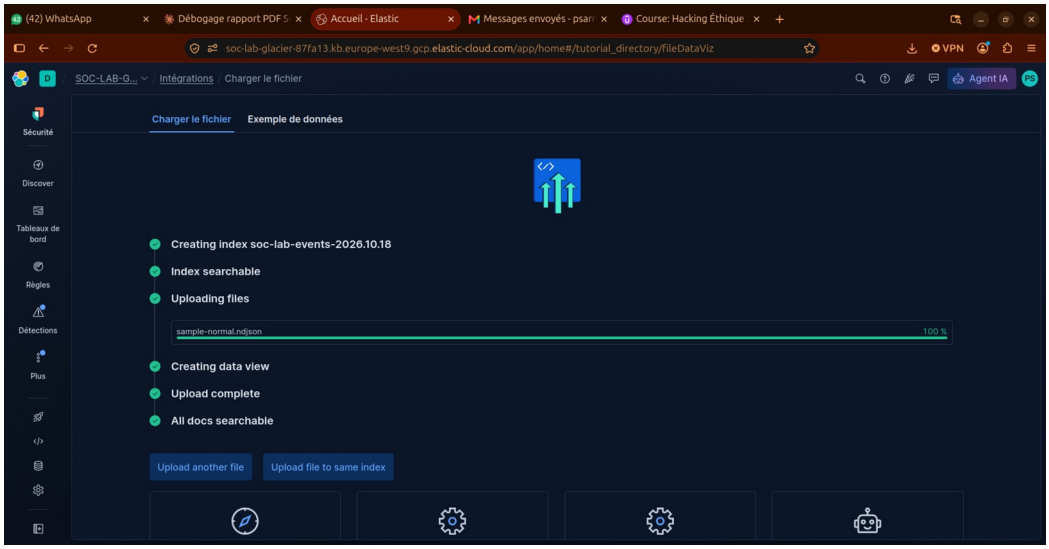
Premier upload et correction d'une erreur de saisie

Lors du premier import (sample-normal.ndjson), après previsualisation du fichier, l'onglet Advanced a été utilise pour nommer l'index. Une confusion s'est produite a cette étape : le champ Data view name a été rempli avec soc-lab-events-2026.10.18, mais le champ Index name, situe plus haut dans le formulaire et distinct du précédent, avait été laisse vide. Kibana a signale l'erreur par un message rouge Data view does not match index name. Après avoir identifie que ces deux champs sont independants (le nom d'index cree physiquement l'index Elasticsearch et doit respecter le pattern soc-

lab-events-* du template ; le nom de Data view est un simple libelle d'affichage, généralement synchronise automatiquement), le champ Index name a été correctement renseigne et l'import a abouti : Creating index, Index searchable, Uploading files 100%, Creating data view, Upload complète.



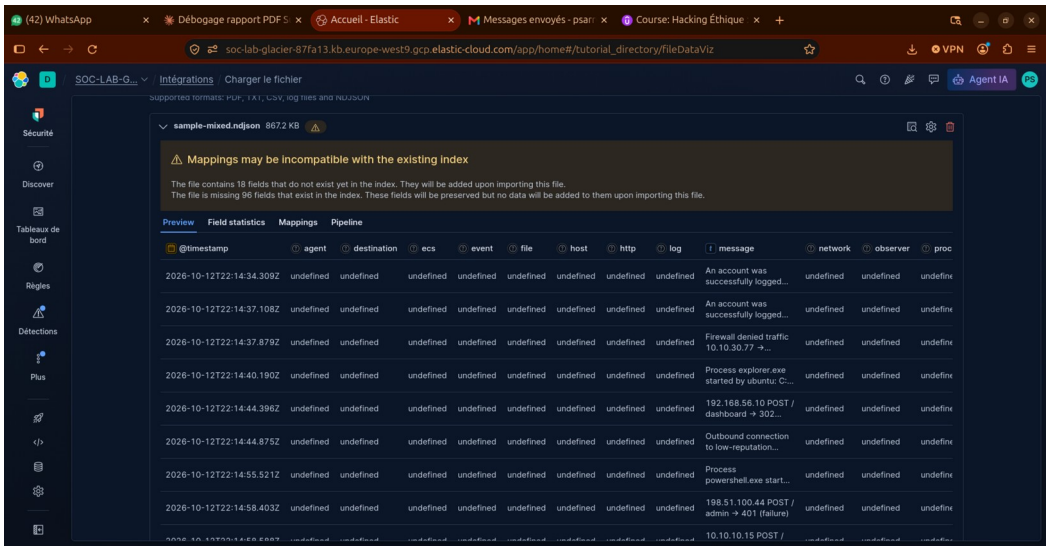
Message d'erreur rouge "Data view does not match index name" : le champ Data view name a été rempli avant le champ Index name, laisse vide.



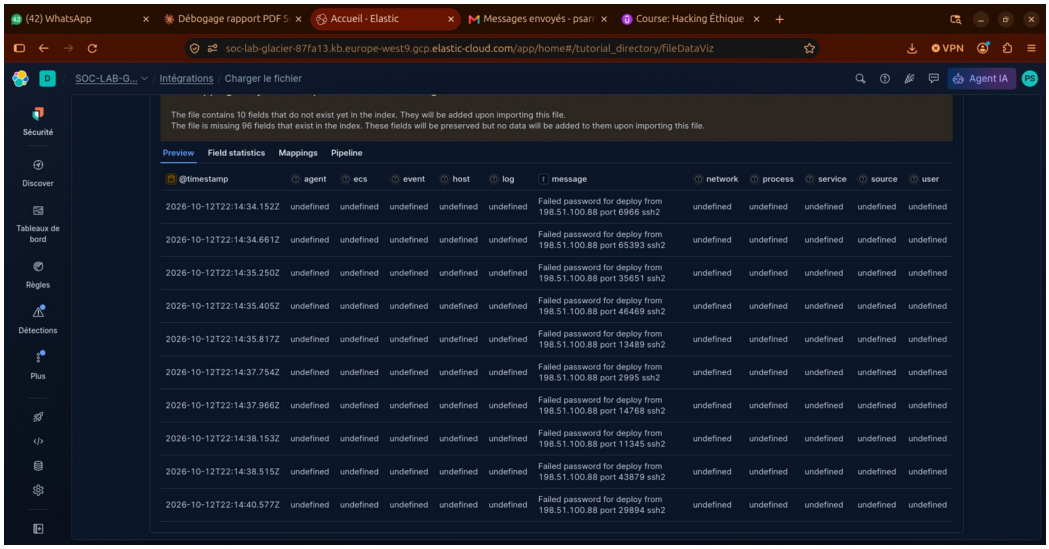
Après correction du champ Index name : Creating index soc-lab-events-2026.10.18, All docs searchable, import réussi.

Imports suivants dans le même index

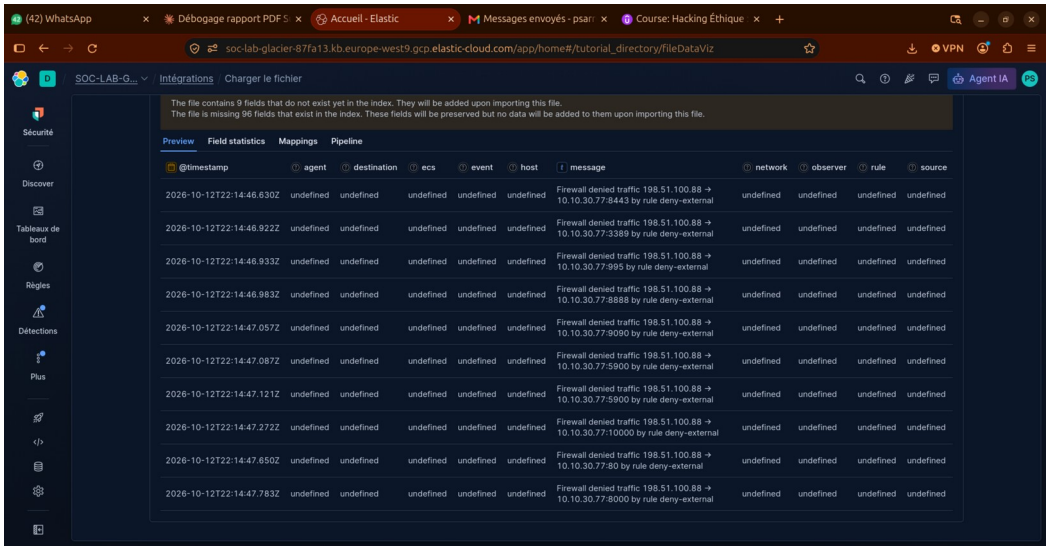
Les quatre autres datasets (sample-mixed, scénario-brute-force, scénario-port-scan, scénario-web-attack) ont été uploades a la suite, dans le même index soc-lab-events-2026.10.18, via le bouton Upload file to same index. Chaque import a affiche un bandeau d'avertissement Mappings may be incompatible with the existing index (par exemple 18 nouveaux champs / 96 champs manquants pour sample-mixed), ce qui n'est pas une erreur : chaque dataset ne remplit naturellement qu'un sous-ensemble des quelque 100 champs ECS predeclares par le template (dynamic: true permettant par ailleurs l'ajout automatique de tout champ réellement nouveau).



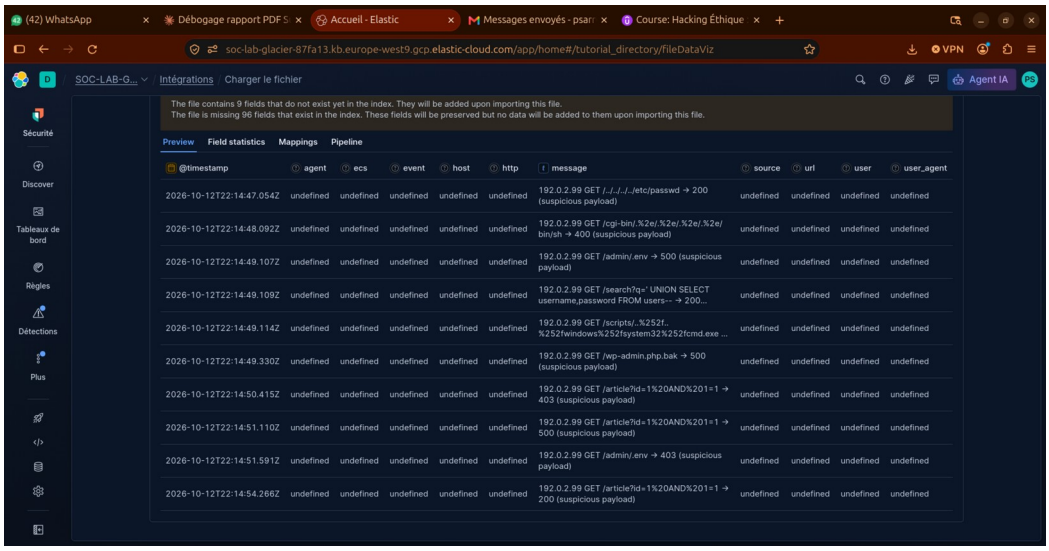
Previsualisation de sample-mixed.ndjson : avertissement "Mappings may be incompatible" (18 champs nouveaux / 96 manquants), non bloquant.



Previsualisation de scénario-brute-force.ndjson : echecs SSH repetes sur le compte deploy, même avertissement de mapping non bloquant.



Previsualisation de scénario-port-scan.ndjson : trafic bloque par la règle deny-external sur FW-EDGE01, même avertissement de mapping.



Previsualisation de scénario-web-attack.ndjson : payloads d'injection (/etc/passwd, UNION SELECT..., traversee de repertoire), même avertissement de mapping.

Pourquoi l'index doit commencer par soc-lab-events- (Q5.1)

C'est la condition nécessaire et suffisante pour que le template applique en Partie II s'active automatiquement au moment de la création de l'index : le pattern index_patterns du template n'accepte que les noms commençant par ce prefixe. Un index nomme differemment serait cree avec le mapping dynamique par defaut d'Elasticsearch, sans beneficier des types explicites (ip, keyword) configures.

Inference de mapping contre template (Q5.2)

L'inference de mapping est une estimation automatique, réalisée document par document au moment de l'upload, qui devine le type de chaque champ a partir des premières valeurs rencontrées (souvent text par defaut pour les chaînes). Le template, a l'inverse, est une declaration explicite et antérieure a toute ingestion : il garantit un typage identique et predictible quel que soit l'ordre ou le contenu des fichiers importés ensuite, ce qui est indispensable pour que les règles de détection (TP4) et les agregations restent fiables sur la durée.

Upload par fichier et flux de production (Q5.3)

Non : l'upload par fichier est explicitement concu par Elastic pour la découverte initiale et l'exploration ponctuelle de données (demonstration, prototypage), pas pour un flux continu. Il impose une action manuelle a chaque fichier, une limite de taille de 100 Mo par fichier, et ne gere ni planification ni retry automatique. Un flux de production réel s'appuierait sur un pipeline automatise (agent Elastic, Logstash, ou l'API _bulk utilisee plus loin dans ce TP, Partie VIII, Defi D).

Point de contrôle 3

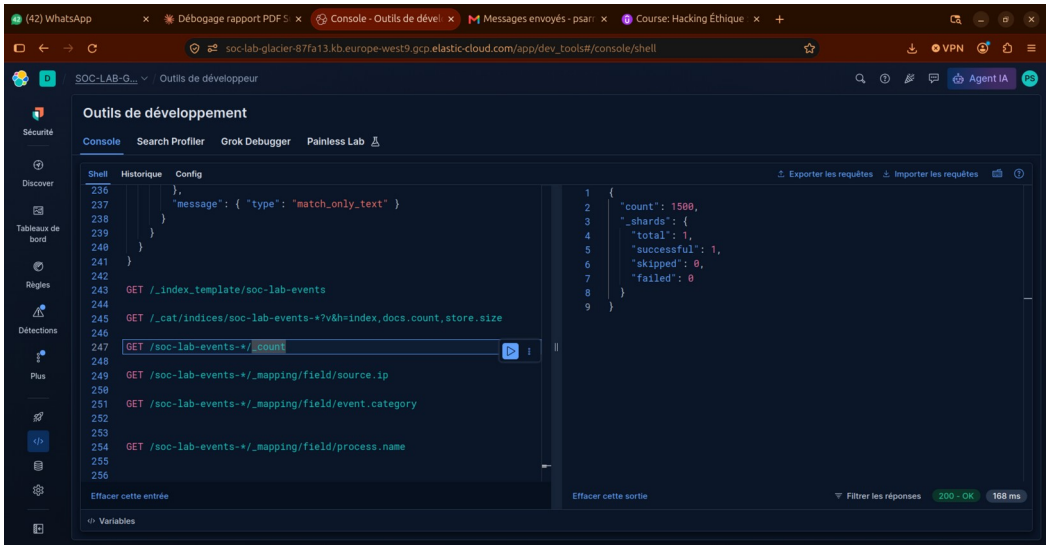
Les cinq datasets (hors tp5-incident.ndjson) sont visibles dans l'apercu de l'index soc-lab-events-2026.10.18, chacun ayant été importe avec succès malgre les avertissements de mapping, non bloquants.

Partie IV : Vérifier l'ingestion (Dev Tools)

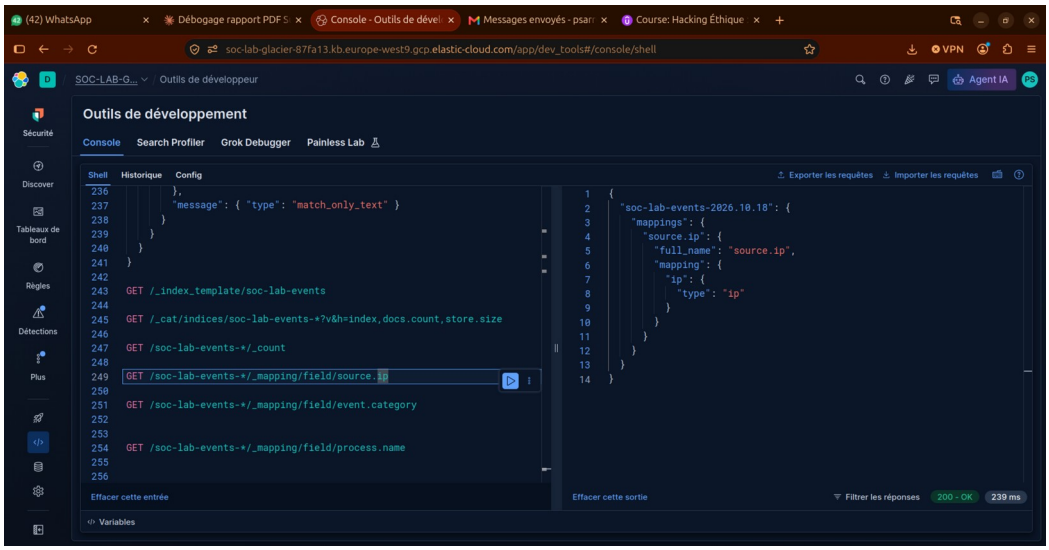
Cinq requêtes de verification ont été exécutées dans Dev Tools après les cinq uploads :

```
GET /_cat/indices/soc-lab-events-*?v&h=index,docs.count,store.size
GET /soc-lab-events-*/_count
GET /soc-lab-events-*/_mapping/field/source.ip
GET /soc-lab-events-*/_mapping/field/event.category
GET /soc-lab-events-*/_mapping/field/process.name
```

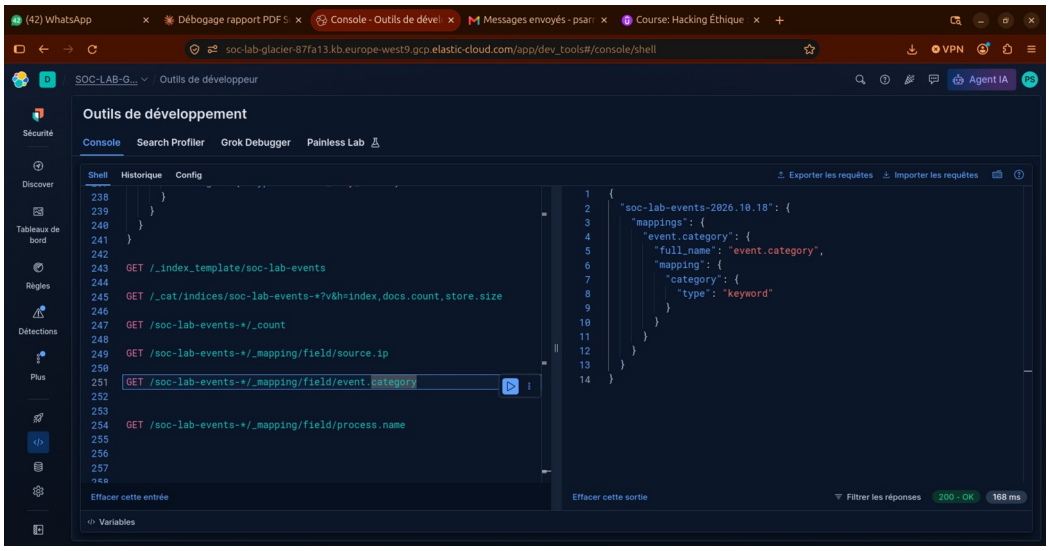
Résultats : l'index unique soc-lab-events-2026.10.18 contient 1500 documents pour une taille de stockage de 689,3 ko, confirmant l'addition exacte des cinq datasets (300 + 1000 + 60 + 80 + 60). Le comptage via /_count confirme également 1500, avec tous les shards en succès.



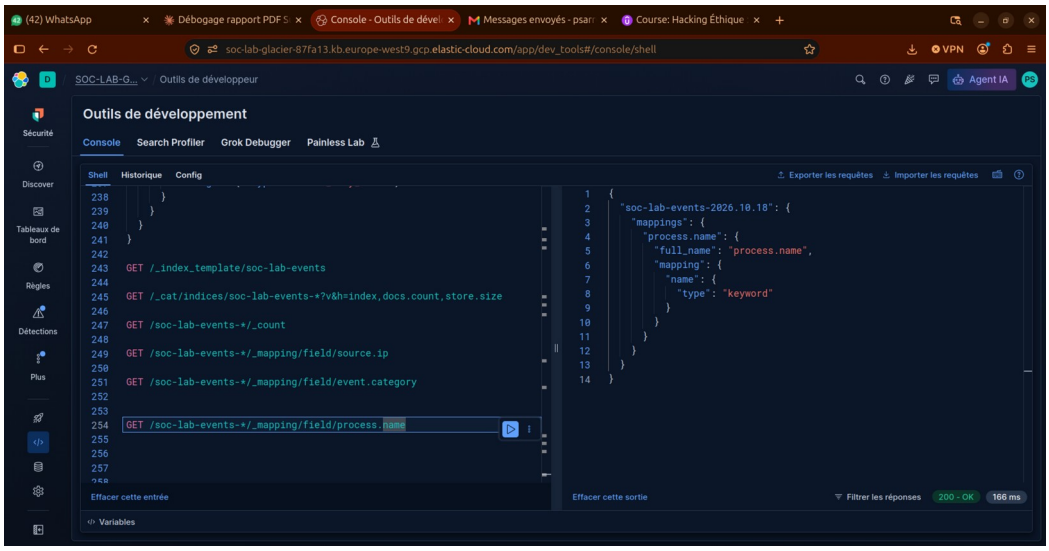
Dev Tools : GET /_cat/indices/soc-lab-events-?v&h=index,docs.count,store.size -> soc-lab-events-2026.10.18, 1500 docs, 689.3kb.*



Dev Tools : GET /soc-lab-events-/_count -> {"count": 1500}, tous les shards en succès.*



Dev Tools : GET .../_mapping/field/event.category -> type keyword confirme.



Dev Tools : GET .../_mapping/field/process.name -> type keyword confirme.

Volume total et nombre d'index (Q6.1)

1500 documents au total, repartis dans un seul index (soc-lab-events-2026.10.18), puisque les cinq uploads ont cible le même index plutôt que cinq index distincts.

Types des trois champs verifies (Q6.2)

Champ	Type confirme
source.ip	ip
event.category	keyword
process.name	keyword

Ces trois types correspondent exactement a ceux declares dans le template de la Partie II, preuve que le template a bien été applique avant le premier import.

Un même index pour plusieurs scénarios (Q6.3)

Oui, un même index peut parfaitement contenir des documents issus de scénarios différents : c'est exactement ce qui a été demontre ici, en uploadant successivement cinq datasets distincts (trafic normal, brute force, scan de ports, attaque web, trafic mixte) dans le même index soc-lab-events-2026.10.18. Elasticsearch ne distingue pas

les documents par leur provenance mais uniquement par leur contenu ; c'est la raison pour laquelle aucun champ du schéma ECS n'indique explicitement le scénario d'origine, l'analyste devant le déduire de l'analyse (point rappele dans le README des datasets en vue du TP5).

Point de contrôle 4

Confirme : source.ip est de type ip, event.category et process.name sont de type keyword, sur un total de 1500 documents dans un index unique.

Partie V : Créer la Data View

La Data View a été créée depuis Stack Management > Data Views > Create data view, avec les paramètres imposes par l'enonce : Name = SOC-LAB Events (simules), Index pattern = soc-lab-events-*, Time field = @timestamp, et surtout un Custom data view ID fixe a soc-lab-events (obtenu en ouvrant le lien Set a Custom data view ID, cache par defaut sous Afficher les paramètres avances).

Un bug d'URL rencontre et corrige

En tentant de naviguer directement vers la page des Data Views par une URL tapee a la main, une double barre oblique s'est glissee dans l'adresse (…elastic-cloud.com//app/management/kibana/dataViews). Kibana a repondu par une page d'erreur JSON brute ({"statusCode":404,"error":"Not Found","message":"Not Found"}) plutôt que par l'interface attendue, le routage serveur ne reconnaissant pas ce chemin malforme. La correction a consiste a retirer la barre oblique en trop ; la navigation a ensuite été privilegiee via le menu lateral Stack Management plutôt que par saisie manuelle d'URL, pour éviter de reproduire ce type d'erreur.

Data View contre index (Q7.1)

L'index est la structure de stockage physique d'Elasticsearch : c'est la ou les documents sont effectivement ecrits et indexes. La Data View est un objet propre a Kibana, une couche d'abstraction qui référence un ou plusieurs index (via un pattern) pour permettre leur exploitation dans l'interface : Discover, visualisations Lens, dashboards. Sans Data View, aucune de ces applications Kibana ne peut interroger les index, même si les données y sont presentes et accessibles via l'API brute (Dev Tools).

Pourquoi un ID personnalise plutôt qu'aléatoire (Q7.2)

Les dashboards construits au TP4 referencent une Data View par son identifiant technique (ID), pas par son nom affiche. Si la Data View avait été créée avec l'ID aléatoire genere par defaut, tout dashboard important plus tard et référence a l'ID attendu (soc-lab-events) se retrouverait vide, faute de correspondance. Fixer l'ID des maintenant evite ce piege silencieux, qui ne se manifeste qu'au moment de l'import des dashboards, bien plus tard dans le fil rouge.

Pourquoi @timestamp est le champ de temps attendu (Q7.3)

@timestamp est le champ temporel standard du schéma ECS, présent de manière identique dans tous les documents, quel que soit leur scénario ou leur module d'origine. C'est ce champ que Kibana utilise pour construire l'histogramme temporel de Discover, pour appliquer une plage de temps (filtre global) et pour trier chronologiquement les résultats. Utiliser un autre champ de date, propre a un seul type d'événement, romprait cette coherence des qu'un dataset différent serait affiche dans la même vue.

Plage de temps et données figees

Une fois la Data View enregistrée, Discover est apparu vide a la première ouverture : la plage de temps par defaut (Last 15 minutes) ne couvre évidemment pas des événements simules et generes a une date fixe (12 octobre 2026). Le passage a Last 7 days a immédiatement fait apparaitre les 1500 documents, avec un pic net dans l'histogramme concentre sur une fenêtre de quelques heures le 12 octobre.

Point de contrôle 5

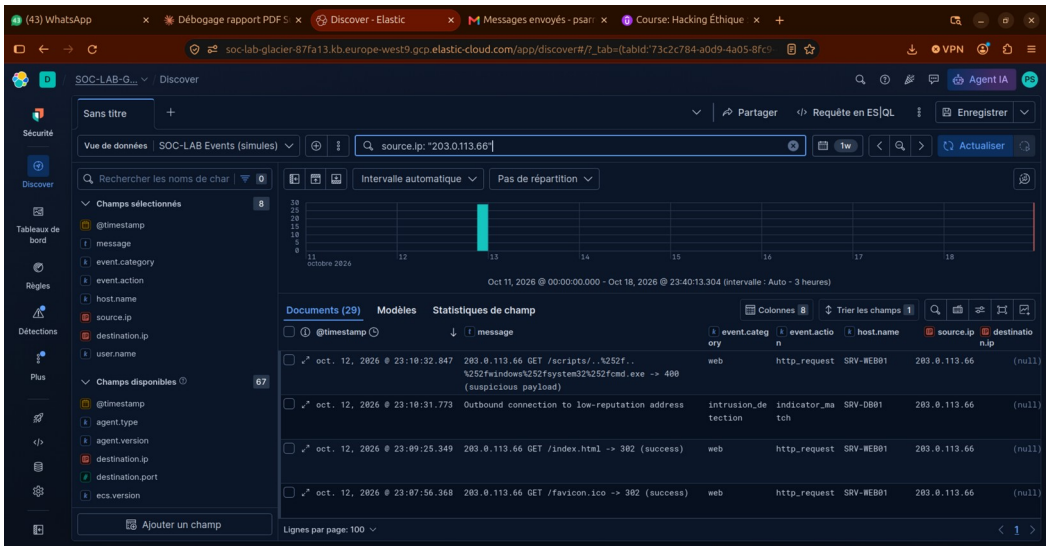
La Data View existe avec l'ID personnalise soc-lab-events, le pattern soc-lab-events-* et le champ temporel @timestamp ; elle affiche les 1500 documents une fois la plage de temps elargie a Last 7 days.

Partie VI : Recherche KQL

8.1 - Premiers filtres

Trois requêtes KQL successives ont été exécutées dans Discover (Data View SOC-LAB Events, plage Last 7 days) :

```
host.name: "SRV-WEB01"                                          -> 488 documents
event.category: "authentication" and event.outcome: "failure"   -> 165 documents
event.category: "web" and http.response.status_code: 403        -> 18 documents
```



Discover, requête event.category: "authentication" and event.outcome: "failure", 165 documents, colonnes event.category/event.action/host.name/source.ip mises en surbrillance.

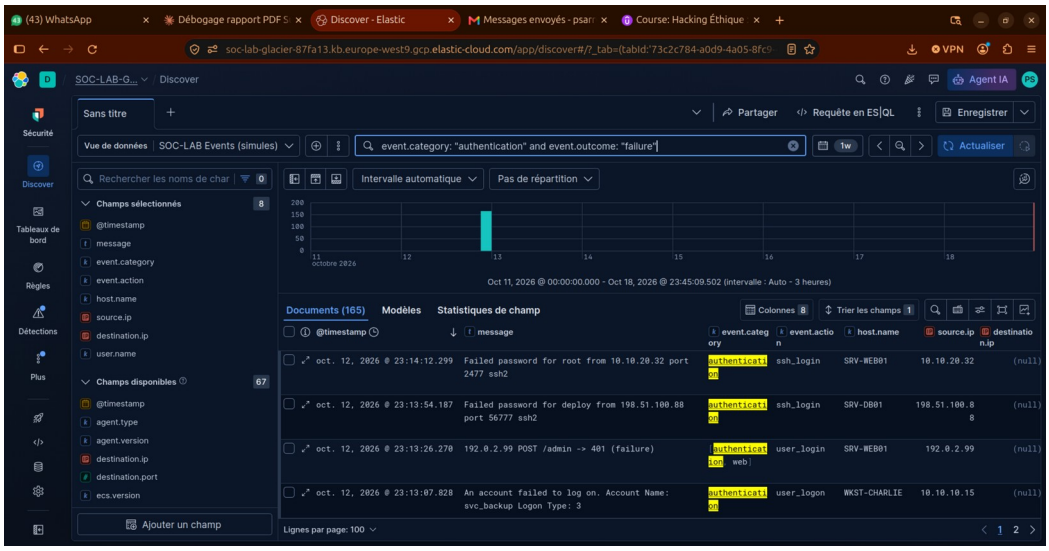
La troisième requête (echecs web 403) a revele des signatures nettes d'attaque applicative contre SRV-WEB01 : tentatives d'injection SQL, de traversee de repertoire et de sondage de fichiers sensibles, en provenance de trois adresses IP distinctes (198.51.100.23, 203.0.113.66, 192.0.2.99).

Volume d'echecs d'authentification sur 7 jours (Q8.1)

165 documents correspondent au filtre event.category: "authentication" and event.outcome: "failure" sur la fenêtre de 7 jours.

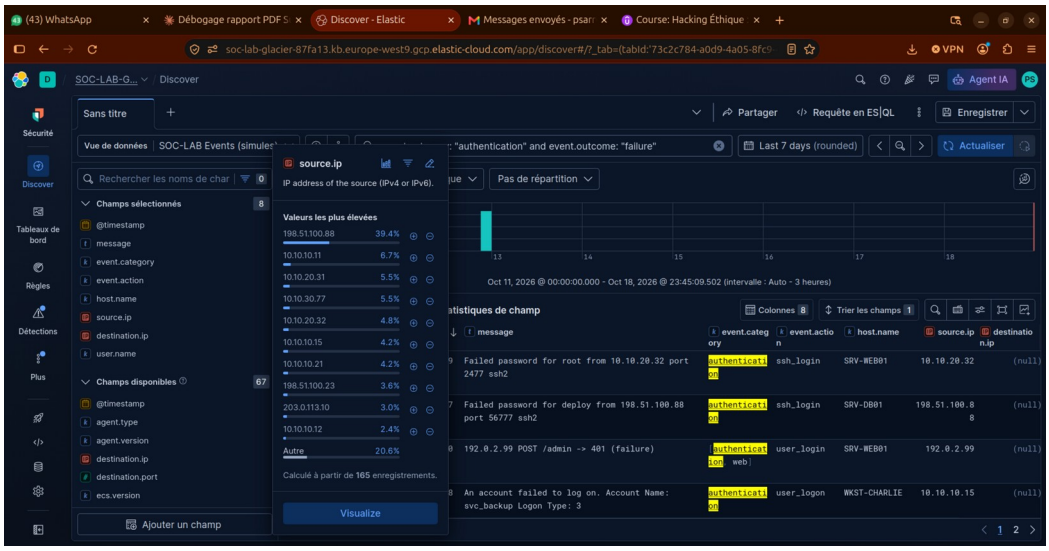
8.2 - Suivre une IP source

Première tentative, erronée : en reprenant l'adresse 203.0.113.66 déjà observee dans les résultats de la requête web/403 précédente (au lieu de la lire specifiquement dans le Top values du filtre d'authentification), la requête source.ip: "203.0.113.66" a certes renvoye 29 documents, mais ceux-ci relevaient des catégories web et intrusion_detection, pas authentication : ce n'était donc pas la bonne IP pour répondre a la question posée.



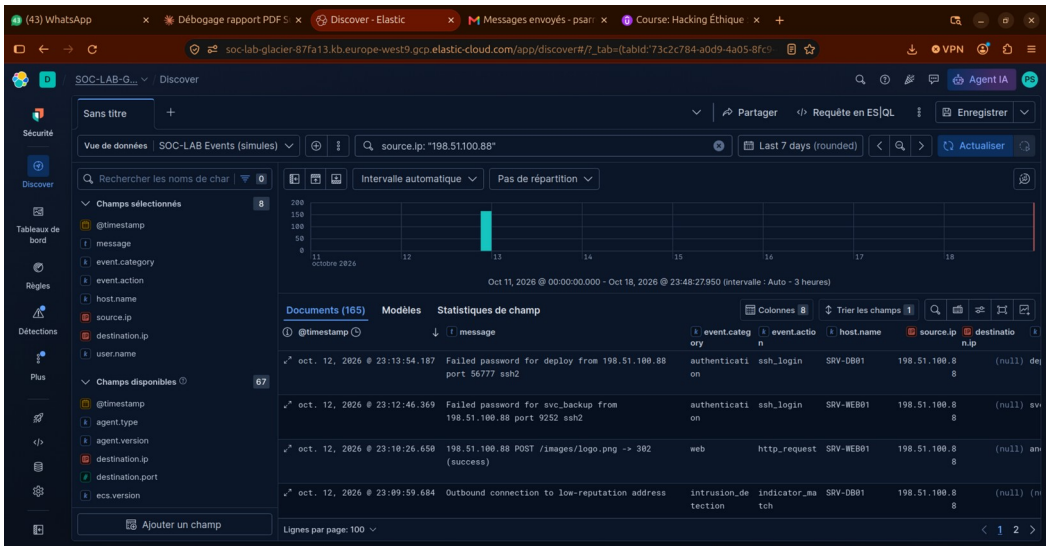
Première tentative (erronée) : source.ip: "203.0.113.66", 29 documents, mais de catégorie web/intrusion_detection et non authentication -- IP reprise par erreur de la requête précédente plutôt que lue dans le Top values du bon contexte.

Méthode corrigée : avec le filtre event.category: "authentication" and event.outcome: "failure" actif (165 documents), un clic sur le champ source.ip dans le panneau de gauche de Discover ouvre son panneau Top values, qui agrege les valeurs les plus frequentes du champ sur le jeu de résultats filtre.

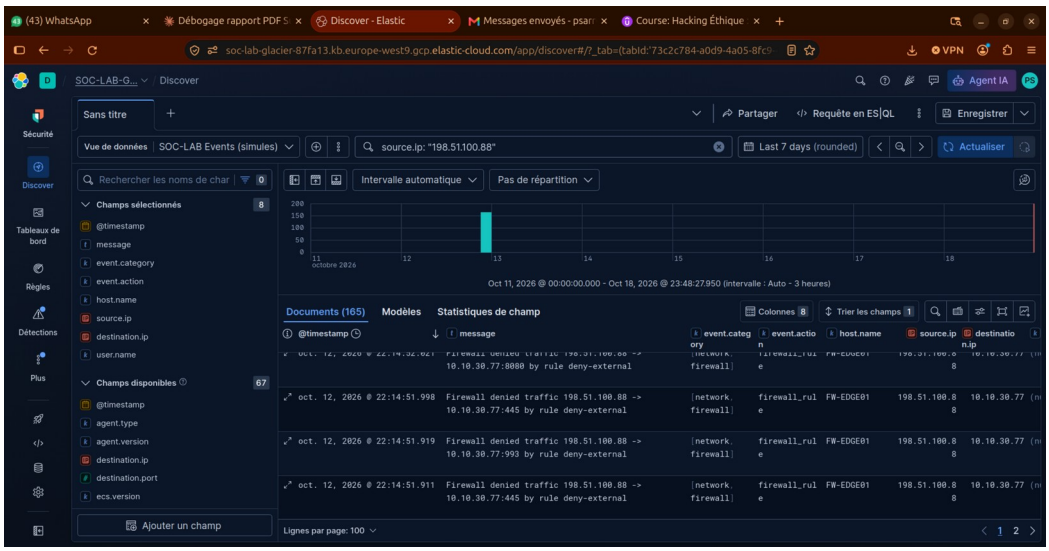


Top values de source.ip sur les 165 echecs d'authentification : 198.51.100.88 concentre 39,4% des événements, très largement devant les IP suivantes (10.10.10.11 a 6,7%).

L'adresse 198.51.100.88 concentre 39,4% des 165 echecs (environ 65 événements), une concentration nette caractéristique d'un brute force automatise. La requête source.ip: "198.51.100.88" (sans filtre de catégorie) montre que cette même IP ne se limite pas a l'authentification : elle touche également les catégories web et intrusion_detection, et genere du trafic bloque par le pare-feu FW-EDGE01.



source.ip: "198.51.100.88" sans filtre de catégorie : echecs SSH sur SRV-DB01 et SRV-WEB01, requêtes web, et une connexion sortante signalée vers une adresse a faible réputation.

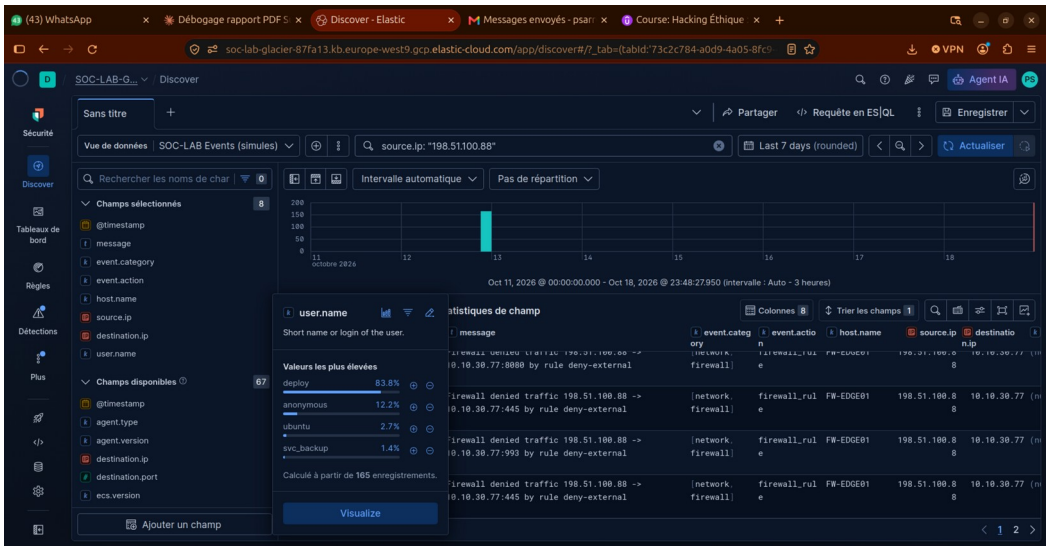


Événements firewall_rule sur FW-EDGE01 : trafic 198.51.100.88 -> 10.10.30.77 bloque sur plusieurs ports (8080, 445, 993) par la règle deny-external, signature d'un balayage de ports complémentaire au brute force.

IP concentrant le plus d'echecs, interne ou externe (Q8.2)

198.51.100.88 concentre 39,4% des 165 echecs d'authentification. Le champ source.geo.country_name s'est revele vide pour l'ensemble du dataset (absence d'enrichissement GeoIP dans les données simulees), la réponse s'appuie donc sur l'analyse de la plage d'adresse : 198.51.100.0/24 est une plage reservee par l'IANA a la documentation et aux tests (RFC 5737, TEST-NET-2), simulant ici une adresse externe au réseau, par opposition aux adresses en 10.0.0.0/8 (RFC 1918, privees/internes) observees ailleurs dans le dataset.

Hotes et comptes cibles par cette IP (Q8.3)



Top values de user.name sur source.ip: "198.51.100.88" : deploy a 83,8%, loin devant anonymous (12,2%), ubuntu (2,7%) et svc_backup (1,4%).

Hotes touches (Top values de host.name) : FW-EDGE01 (52,1%, trafic bloque/journalise), SRV-DB01 (39,4%, cible principale du brute force SSH), SRV-WEB01 (6,1%), SRV-FILE01 (1,2%), WKST-ALICE et kali-attacker (0,6% chacun). Comptes cibles (Top values de user.name) : deploy, très largement majoritaire a 83,8%, suivi d'anonymous (12,2%), ubuntu (2,7%) et svc_backup (1,4%). Le compte deploy, compte de service utilise pour l'automatisation, est donc la cible privilegiee de cette campagne, un choix coherent avec un brute force automatise cherchant des identifiants faibles ou reutilises sur des comptes techniques plutôt que sur des comptes personnels.

8.4 - Séquence EQL (optionnel)

Cette requête optionnelle, visant a détecter une séquence échec puis succès d'authentification pour une même IP dans une fenêtre de 15 minutes (POST /soc-lab-events-*/_eql/search avec une clause séquence by source.ip), n'a pas été exécutée faute de temps disponible en fin de seance. Elle reste une piste d'approfondissement naturelle : son résultat determinerait si le brute force observe a fini par aboutir a une connexion réussie, ce qui ferait basculer l'incident d'une simple tentative a une compromission averee.

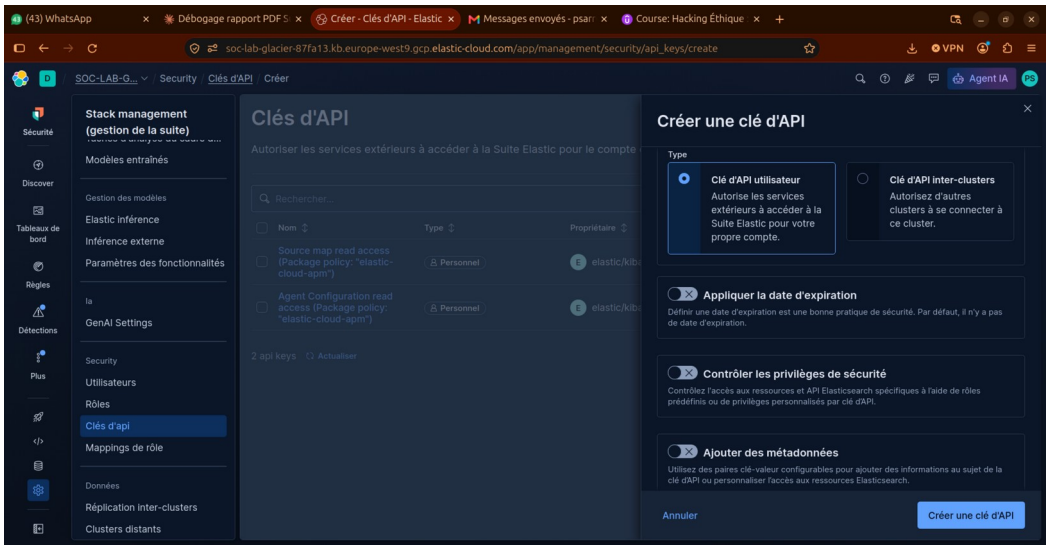
Point de contrôle 6

Des requêtes KQL a deux conditions ont été ecrites et exécutées avec succès, et une adresse IP source suspecte (198.51.100.88) a été profilee de bout en bout : volume, hotes cibles, comptes vises, et nature probable (externe) de l'adresse.

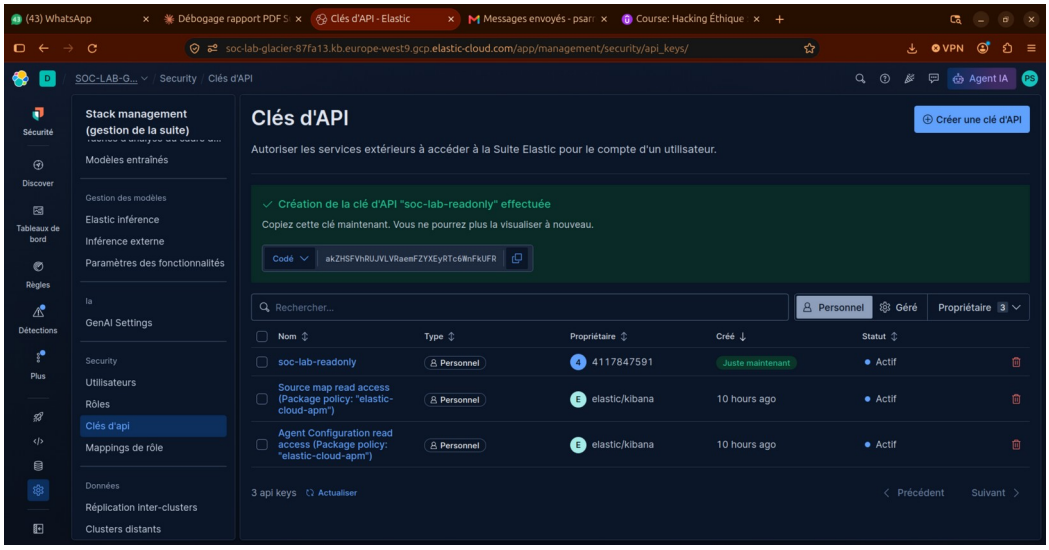
Partie VII : Cle API restreinte

La cle a été créée depuis Stack Management > Security > Clefs d'API > Créer une cle d'API, avec le type Cle d'API utilisateur (par defaut, et correct pour ce cas d'usage), le nom soc-lab-readonly, et l'option Controler les privileges de sécurité activee pour y coller le descripteur de rôle suivant, restreignant la cle au strict nécessaire :

```
{
  "soc-lab-readonly-rôle": {
    "cluster": [],
    "indices": [
      {
        "names": ["soc-lab-events-*"],
        "privileges": ["read", "view_index_metadata"]
      }
    ]
  }
}
```



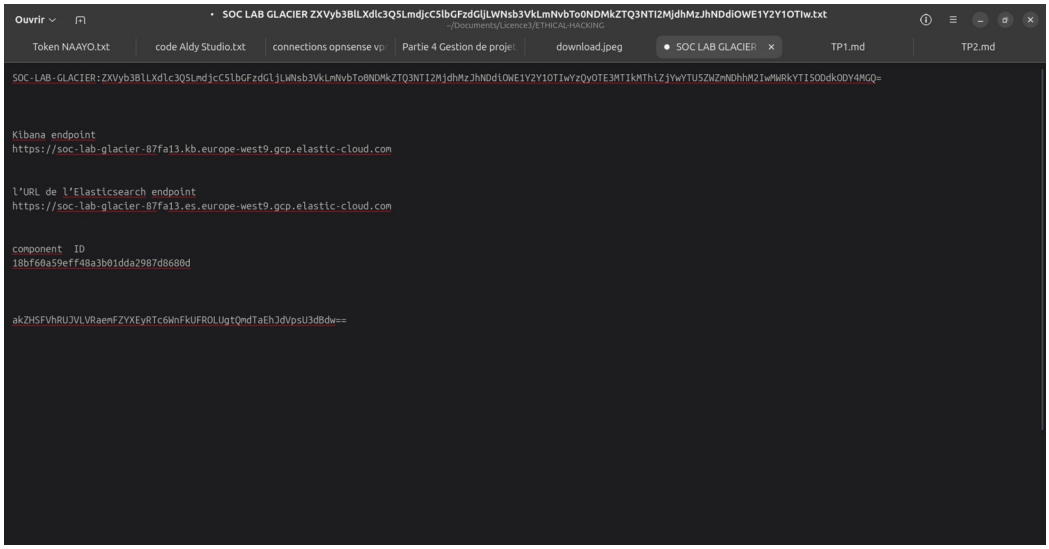
Formulaire de création de cle API : type Cle d'API utilisateur, section Controler les privileges de sécurité disponible.



Confirmation de création : "soc-lab-readonly" listee parmi les clés API, statut Actif, avec le message rappelant que la cle en clair ne sera plus jamais reaffichee.

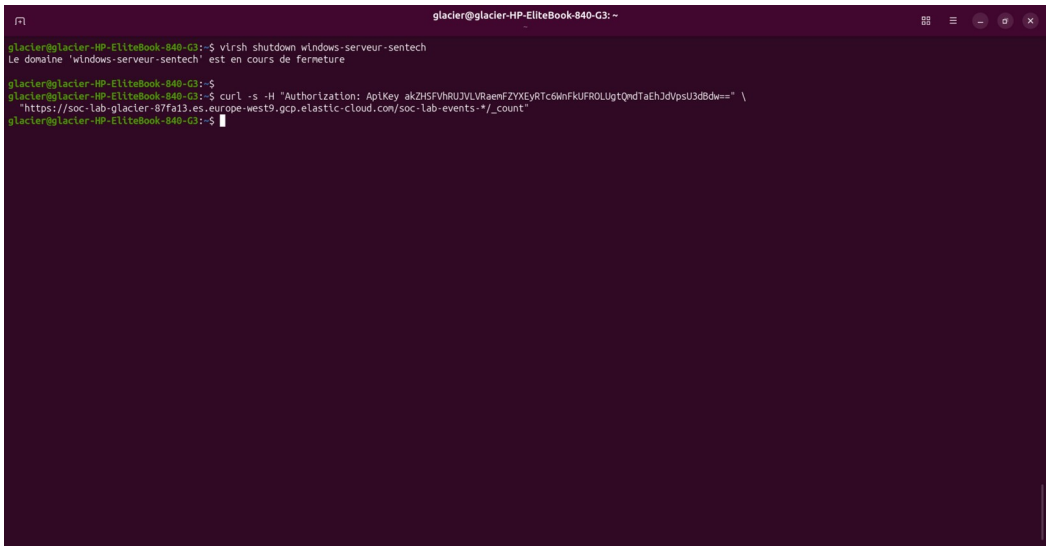
Test de la cle en lecture

Une première tentative d'accès, effectuée en ouvrant directement l'URL de l'endpoint Elasticsearch dans le navigateur plutôt qu'en ligne de commande, a declenche la fenêtre d'authentification HTTP Basic native du navigateur : attendu, puisque le navigateur reagit a un code 401 renvoye par Elasticsearch en proposant sa propre invite de connexion, independamment de toute cle API. Le test a été repris correctement en terminal, avec curl et l'en-tête Authorization: ApiKey.



Fenêtre d'authentification HTTP Basic du navigateur, déclenchée par une ouverture directe de l'URL Elasticsearch -- confusion clarifiee, le test se fait en ligne de commande, pas dans le navigateur.

Un premier essai en curl avec l'option -s (mode silencieux) n'a affiche aucune sortie, ni résultat ni erreur -- cette option supprime en effet aussi bien la barre de progression que les messages d'erreur curl, rendant le diagnostic impossible en cas d'échec réseau transitoire.

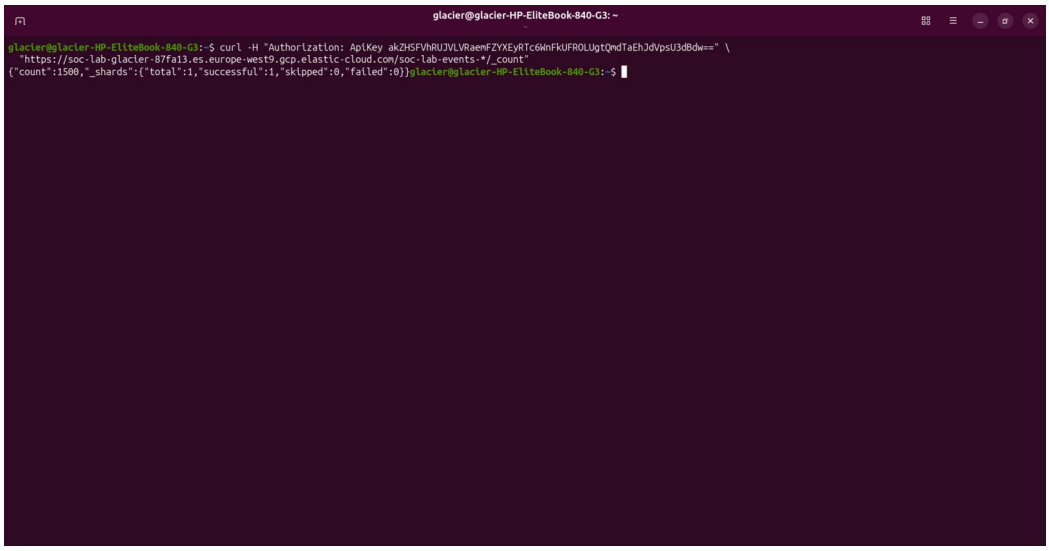


curl -s sans sortie visible : le mode silencieux masque aussi bien un succès vide qu'une erreur réseau.

La commande relancee sans -s a confirme le bon fonctionnement de la cle :

```
curl -H "Authorization: ApiKey <cle_encodee>" \
  "https://soc-lab-glacier-87fa13.es.europe-west9.gcp.elastic-cloud.com/soc-lab-events-
*/_count"

{"count":1500,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}
```



Réponse en clair : {"count":1500,...}, confirmant que la cle en lecture seule retrouve bien les 1500 documents indexes.

Verification du moindre privilege

Pour vérifier concretement que la restriction fonctionne, une tentative de suppression d'index a été effectuée avec la même cle :

```
curl -X DELETE -H "Authorization: ApiKey <cle_encodee>" \
  "https://soc-lab-glacier-87fa13.es.europe-west9.gcp.elastic-cloud.com/soc-lab-events-2026.10.18"
```

Elasticsearch a refuse l'action avec un code 403 et un message explicite : action [indices:admin/delete] is unauthorized for API key ... this action is granted by the index privileges [delete_index,manage,all]. La preuve est directe : la cle ne possede ni delete_index, ni manage, ni all, uniquement read et view_index_metadata, exactement comme configure.



curl -X DELETE refuse avec un code 403 security_exception : preuve concrete du moindre privilege applique a la cle soc-lab-readonly.

Pourquoi ne pas reutiliser le compte elastic (Q9.1)

Le compte elastic est un superutilisateur avec des droits d'administration totaux (création/suppression d'index, gestion des utilisateurs, configuration du cluster). Un analyste SOC n'a besoin que de lire les données déjà indexees. En cas de compromission des identifiants ou du poste de l'analyste, un accès elastic compromis met en peril l'ensemble de la plateforme, alors qu'une cle en lecture seule compromise ne permet,

dans le pire des cas, que de consulter des logs déjà presents -- demontre concretement ci-dessus par l'échec de la tentative de suppression.

Cle API contre utilisateur (Q9.2)

Un utilisateur (comme elastic) est une identite humaine, avec nom et mot de passe, destinée a une session interactive dans Kibana. Une cle API est une credential machine-a-machine : sans session interactive, restreignable finement par privileges et par pattern d'index, assortie d'une date d'expiration optionnelle, et surtout revocable individuellement sans affecter les comptes utilisateurs. C'est le format adapte aux scripts et outils automatises, comme demontre par le test curl ci-dessus.

Revocation d'une cle compromise (Q9.3)

Depuis Stack Management > Security > Clefs d'API, un clic sur l'icone de suppression (poubelle) en bout de ligne revoque immédiatement la cle correspondante : toute requête l'utilisant ensuite recoit un code 401 Unauthorized.

Point de contrôle 7

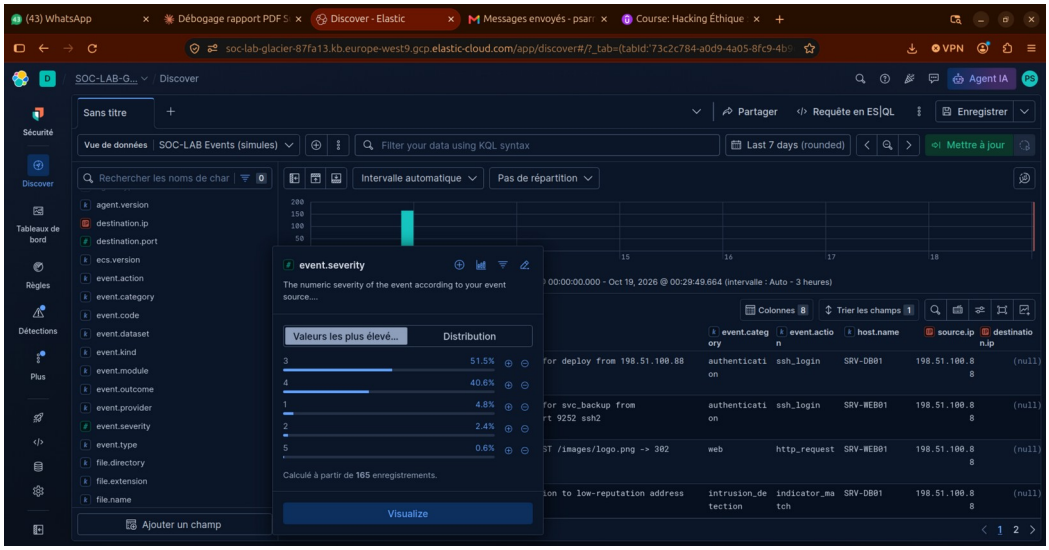
La cle soc-lab-readonly existe, répond correctement en lecture (count: 1500) et refuse explicitement toute action d'écriture (403 sur une tentative de suppression), confirmant un accès strictement en lecture seule.

Partie VIII : Defis (autonomie)

Les quatre defis proposes par l'enonce ont été realises, depassant le minimum de deux demande par la checklist de fin de TP.

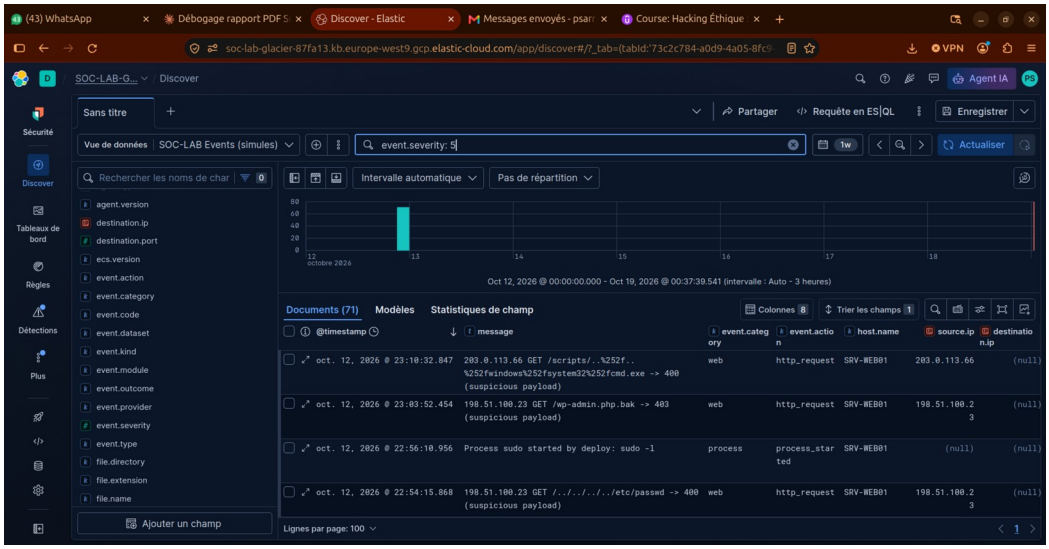
Defi A : le plus grave (event.severity maximal)

Le champ event.severity va de 1 a 5 dans ce dataset. Son panneau Top values indique une distribution de 51,5% (severite 3), 40,6% (severite 4), 4,8% (severite 1), 2,4% (severite 2) et seulement 0,6% pour la severite maximale (5) -- un sous-ensemble volontairement rare, cible via la requête event.severity: 5.

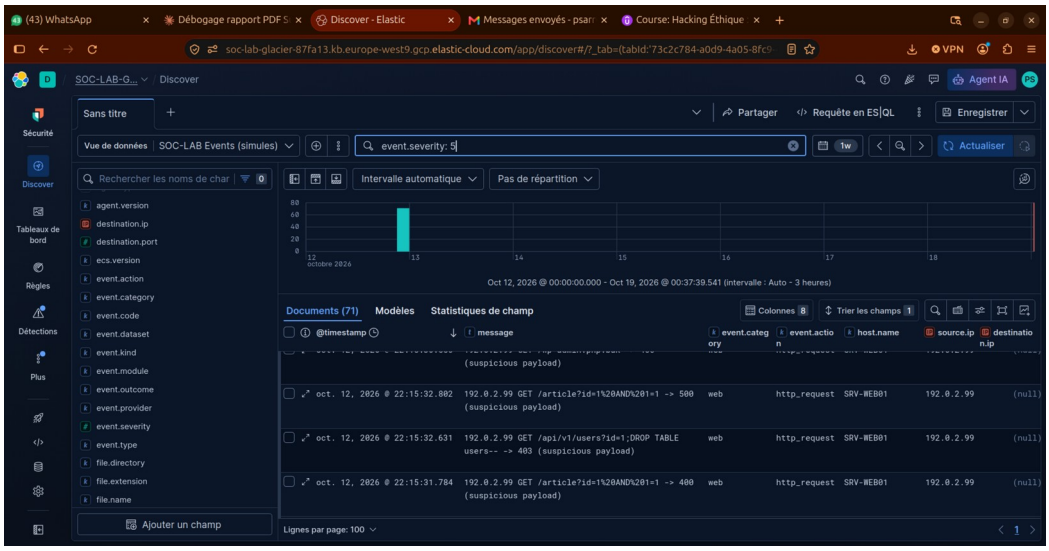


Distribution d'event.severity sur l'ensemble des documents : la severite 5 ne représente qu'une faible part des événements, signe d'une classification selective.

71 documents correspondent a event.severity: 5. La grande majorite est constituee d'attaques web contre SRV-WEB01 (injection SQL, traversee de repertoire, sondage de fichiers sensibles) menees par trois adresses externes distinctes (203.0.113.66, 198.51.100.23, 192.0.2.99).



Premiers résultats event.severity: 5 : tentative d'exécution de cmd.exe par traversee de repertoire, sondage de wp-admin.php.bak, et lecture de /etc/passwd, tous contre SRV-WEB01.

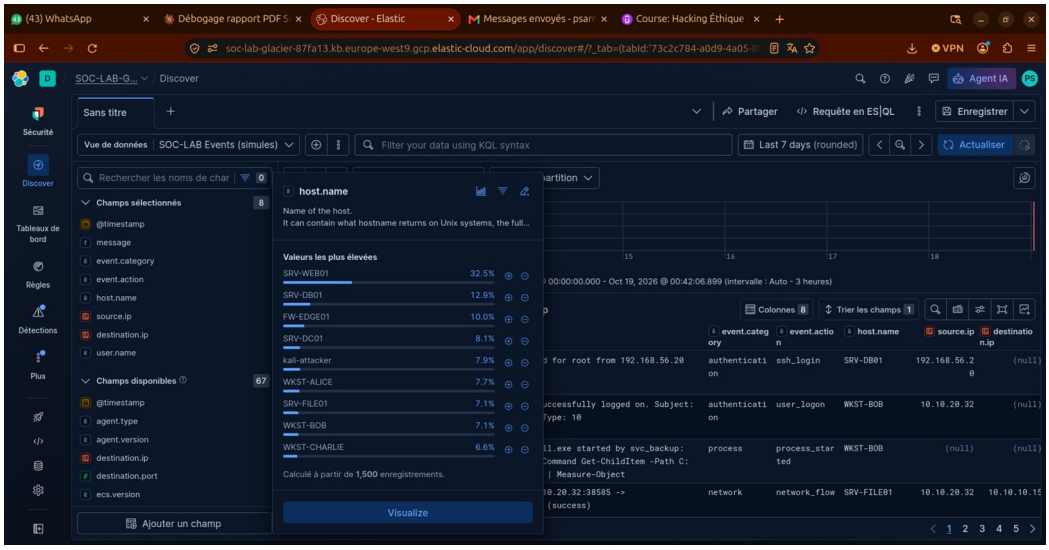


Suite des résultats : injections SQL explicites (DROP TABLE users--, id=1%20AND%201=1) depuis 192.0.2.99 contre SRV-WEB01.

Un seul événement de severite 5 detonne dans cet ensemble : "Process sudo started by deploy: sudo -l" sur SRV-WEB01, classe event.category: process. Sa presence au même niveau de severite que des attaques web caracterisees en fait un excellent cas d'etude pour le Defi C, traite plus loin.

Conclusion du Defi A : l'événement de severite maximale la plus representative du dataset est une campagne d'injection SQL et de traversee de repertoire contre SRV-WEB01, menée depuis plusieurs adresses IP externes sur une fenêtre concentree d'environ une heure, le 12 octobre en soiree.

Defi B : inventaire des hotes



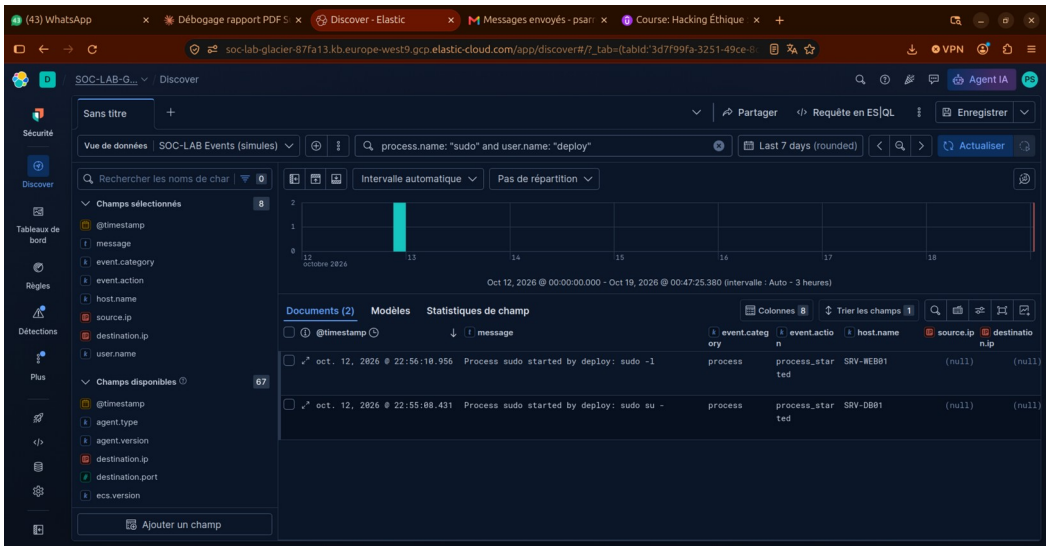
Top values de host.name calcule sur l'ensemble des 1500 documents : 9 valeurs distinctes totalisant environ 100% des événements, sans catégorie "Autre" residuelle.

9 valeurs distinctes de host.name ont été denombrees sur les 1500 documents (7 jours) : SRV-WEB01 (32,5%), SRV-DB01 (12,9%), FW-EDGE01 (10,0%), SRV-DC01 (8,1%), kali-attacker (7,9%), WKST-ALICE (7,7%), SRV-FILE01 (7,1%), WKST-BOB (7,1%) et WKST-CHARLIE (6,6%). Ces pourcentages totalisant environ 100% sans catégorie residuelle Autre, le decompte est exhaustif sans necessiter d'agregation cardinality supplémentaire via Dev Tools. Le parc simule reconstitue ainsi quatre serveurs (web, base de données, fichiers, controleur de domaine), un pare-feu perimetrique, trois

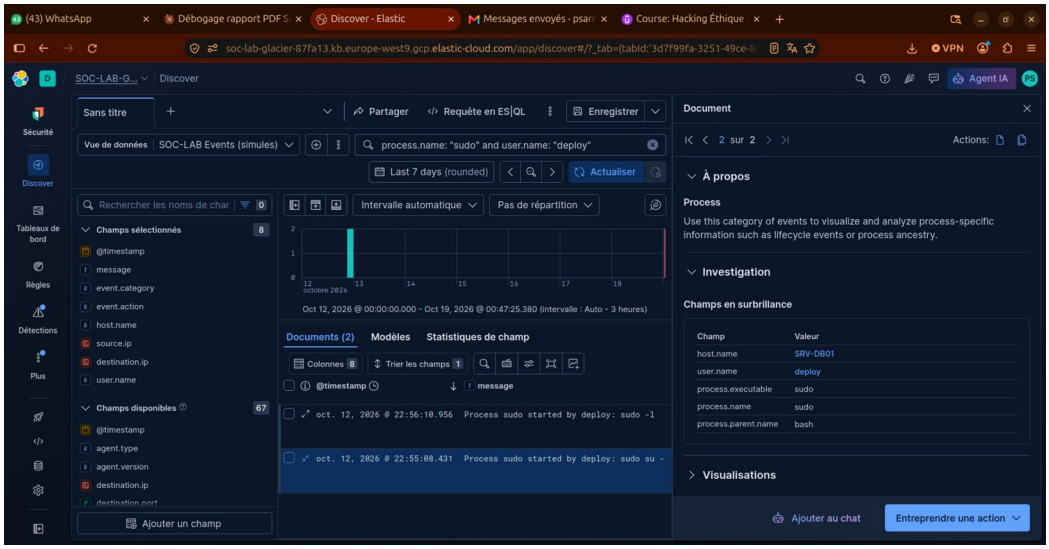
postes utilisateurs, et une machine nommee explicitement kali-attacker, correspondant a la machine de l'attaquant dans le scénario pédagogique.

Defi C : faux positif

Deux événements process ont été isoles avec la requête process.name: "sudo" and user.name: "deploy" : sudo -l sur SRV-WEB01 (22:56:10) et sudo su - sur SRV-DB01 (22:55:08), tous deux classes severite 5.



Les deux événements sudo du compte deploy : sudo -l sur SRV-WEB01 et sudo su - (plus sensible) sur SRV-DB01.



Document developpe de l'événement sudo su - sur SRV-DB01 : process.parent.name vaut bash, signature d'une session shell normale plutôt que d'un processus d'exploitation.

A première vue, ces événements ressemblent a une elevation de privileges suspecte : compte de service deploy (déjà identifie comme cible du brute force SSH en Partie VI), commande sensible (sudo su - ouvre un shell root complet), severite maximale. Plusieurs éléments renversent pourtant cette première lecture. D'abord la chronologie : ces deux sudo ont lieu a 22h55-22h56, soit avant le debut des tentatives de brute force observees sur ce même compte (23h09-23h14) -- un sudo consecutif a une compromission réussie serait au contraire attendu après un succès d'authentification, pas quinze a vingt minutes avant la première tentative d'échec. Ensuite le detail technique : process.parent.name vaut bash, signature d'une session shell interactive normale, et non d'un processus d'exploitation ou d'un payload injecte. Enfin le

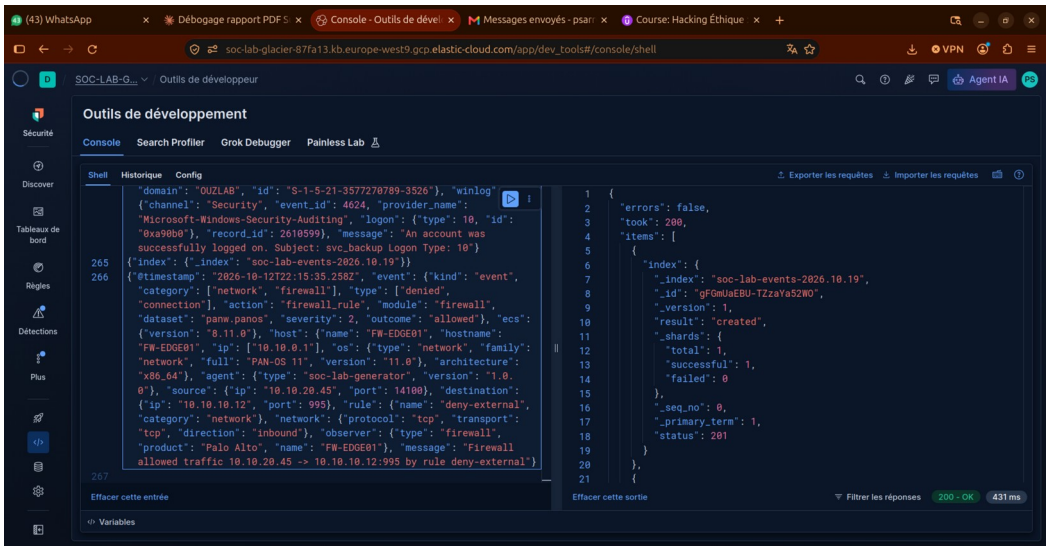
contexte : ce schéma est coherent avec un script de déploiement automatise utilisant ce compte de service pour des tâches d'administration routinieres.

Conclusion du Defi C : l'événement sudo su - du compte deploy sur SRV-DB01 est un faux positif plausible, une activité d'administration legitime survenant avant l'attaque et portée par un compte dont le nom, facilement devinable, en fait neanmoins une cible privilegiee pour un attaquant -- un point a signaler dans les recommandations de durcissement (renommage ou rotation des identifiants des comptes de service standards).

Defi D : ingestion scriptee via _bulk

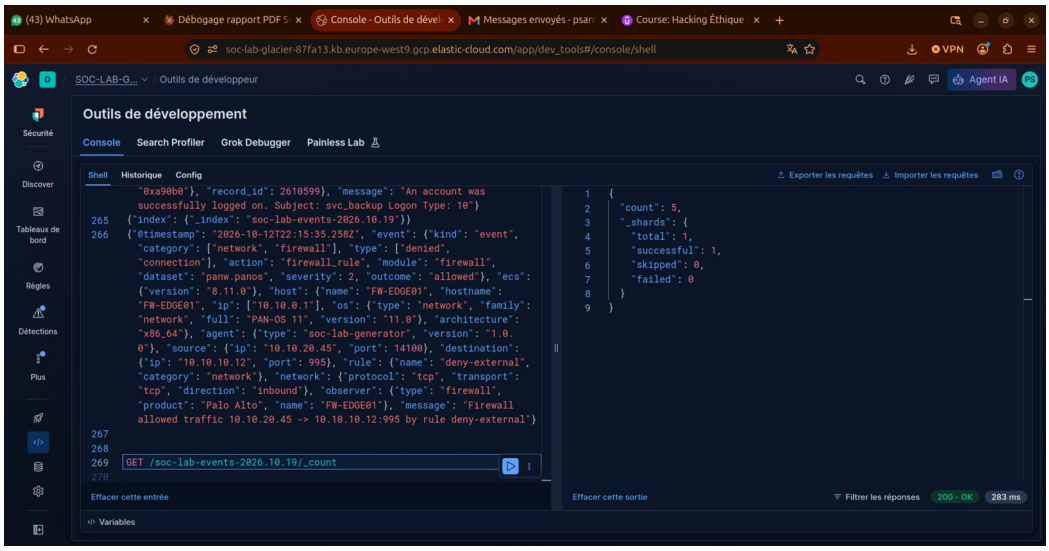
Cinq événements réels, extraits des premières lignes de sample-normal.ndjson, ont été convertis au format _bulk de l'API Elasticsearch. Ce format exige une ligne d'action-métadonnée avant chaque document (par exemple {"index": {"_index": "..."}}), contrairement au format NDJSON source ou chaque ligne est directement un document. La cible choisie, soc-lab-events-2026.10.19 (date du jour de l'exercice), respecte le pattern soc-lab-events-* du template applique en Partie II, qui s'y applique donc automatiquement -- une demonstration concrete, en conditions réelles, de la réponse a la question Q4.3.

```
POST /_bulk
{"index": {"_index": "soc-lab-events-2026.10.19"}}
{"@timestamp": "2026-10-12T22:14:35.328Z", "event": {...}, "host": {"name": "WKST-
BOB", ...}, ...}
{"index": {"_index": "soc-lab-events-2026.10.19"}}
{"@timestamp": "2026-10-12T22:14:56.785Z", "event": {...}, "host": {"name": "SRV-
WEB01", ...}, ...}
... (5 paires action/document au total)
```



Réponse de POST /_bulk : "errors": false, 5 éléments avec "result": "created" et "status": 201 chacun.

La réponse confirme errors: false et un result: created pour chacun des 5 documents (status 201). Une verification finale via GET /soc-lab-events-2026.10.19/_count renvoie count: 5, confirmant que l'ingestion scriptee a parfaitement réussi, sans dupliquer les 1500 documents déjà analyses dans l'index principal.



GET /soc-lab-events-2026.10.19/_count : {"count": 5}, confirmant l'ingestion réussie des 5 documents via l'API _bulk.

Point de contrôle defis

Les quatre defis ont été menes a terme : identification de la campagne d'attaque web de severite maximale (Defi A), inventaire exhaustif des 9 hotes du parc simule (Defi B), qualification argumentee d'un faux positif d'administration (Defi C), et demonstration réussie d'une ingestion scriptee via l'API _bulk (Defi D).

Conclusion

Ce TP2 a permis d'alimenter de bout en bout la plateforme SOC-LAB-GLACIER, jusque-la vide : application d'un template d'index ECS garantissant des mappings corrects, upload des cinq datasets simules via Kibana, verification systématique de l'ingestion dans Dev Tools, création d'une Data View a identifiant personnalise (condition nécessaire aux dashboards du TP4), premières recherches KQL permettant de profiler une campagne de brute force SSH jusqu'a son compte et ses hotes cibles, et création d'une cle API en lecture seule appliquant concretement le principe de moindre privilege.

Plusieurs obstacles réels ont jalonne ce TP et ont été documentes tels qu'ils se sont presentes plutôt que masques : une confusion entre les champs Index name et Data view name au premier upload, un bug de double barre oblique dans une URL tapee a la main provoquant une erreur 404, une reutilisation erronée d'une adresse IP lue dans le mauvais contexte lors du suivi d'une IP source, une commande curl en mode silencieux masquant temporairement un résultat, et une Data View incorrecte sélectionnée par inadvertance au moment d'examiner l'événement sudo du Defi C. Chacune de ces erreurs a été identifiée, expliquee et corrigée en suivant une méthode reproductible, ce qui constitue en soi une competence d'analyste SOC aussi importante que la maitrise technique des outils.

Les quatre defis d'autonomie ont permis d'aller au-delà du strict minimum demande : une campagne d'attaque web caracterisee (Defi A), un inventaire exhaustif du parc (Defi B), une première qualification argumentee d'un faux positif (Defi C), et la demonstration d'une méthode d'ingestion automatisable par script (Defi D), en réponse directe a la limite de l'upload manuel identifiée en Partie III (Q5.3).

La plateforme contient désormais 1505 documents repartis sur deux index (1500 dans soc-lab-events-2026.10.18, 5 dans soc-lab-events-2026.10.19), une Data View correctement configurée, et une première piste d'activité suspecte solidement etayee (l'adresse 198.51.100.88 et le compte deploy). C'est sur cette base que s'appuiera le TP3, consacre a la collecte de données réelles via un agent Elastic connecte au Fleet Server hébergé.